

Compiladores, 2025/2026

Trabalho prático, parte 3 – Fava: Funções, variáveis locais –

Fernando Lobo

1 Introdução

Nesta parte do trabalho prático, vamos estender a linguagem Fava para permitir a declaração e uso de funções, bem como a declaração e uso de variáveis locais.

Como consequência, a tabela de símbolos terá de suportar enquadramentos encaixados (nested scopes) como foi explicado nas aulas teóricas e ser capaz de guardar informação relevante para todo o tipo de símbolos (variáveis globais, variáveis locais, nomes de funções, argumentos de funções).

Devido à introdução de funções, a linguagem Fava terá de permitir mais dois tipos de instrução, nomeadamente:

- chamada de função
- return

Por outro lado, teremos de acrescentar novas instruções à máquina virtual de modo a poder suportar a execução de funções.

A chamada de uma função poderá aparecer no programa fonte antes da declaração da função, sem que isso cause qualquer erro.

2 Alterações à linguagem

2.1 Programa

Um programa em Fava passa agora a ser constituído por uma sequência de zero ou mais declarações de variáveis globais, seguido de uma sequência de uma ou mais

declarações de funções, sendo que uma dessas funções terá de se chamar **main**, o ponto de entrada para a execução do programa.

2.2 Declaração de função

Uma declaração de função é feita indicando a palavra reservada **function**, seguido do nome da função, seguido de um parêntesis curvo a abrir, seguido de uma lista de definição de argumentos separados por vírgula, seguido de um parêntesis curvo a fechar, e no caso da função retornar um valor deve ainda ser seguido do símbolo **->**, seguido do tipo de retorno (que pode ser um dos 4 tipos base da linguagem: **integer**, **real**, **bool** ou **string**), seguido de um bloco.

No caso da função não retornar um valor, não se coloca o símbolo **->** nem o tipo de retorno, ficando implícito que se trata de uma função **void**.

A definição de argumento é especificada indicando o tipo, seguido do nome do argumento. O nome da função e argumento(s) obedecem às mesmas regras lexicais usadas para as declarações de variáveis. Em Fava a passagem de parâmetros é feito por valor.

Eis um exemplo com a declaração de 2 funções:

```
function fatorial( integer n ) -> integer
{
    if (n = 0) return 1;
    return n * fatorial(n-1);
}

function xpto( integer a, integer b )
{
    print a + b;
}
```

O compilador deverá fazer a verificação de tipos. Quando uma função é definida como retornando um valor de determinado tipo, o compilador deve garantir que de facto a função retorna um valor, e que esse valor tem um tipo de dados consistente com o tipo de dados declarado.

Consistência de tipos significa que os tipos têm de ser iguais, havendo apenas a excepção de algo declarado como sendo do tipo **real** poder receber um valor do tipo **integer**, devendo neste caso o compilador emitir código para fazer a conversão usando a instrução **itod** da máquina virtual.

No caso da função não retornar um valor, a existência de uma instrução de (**return** explícito por parte do programador é opcional, à semelhança de que acontece em C e Java.

2.3 Novas instruções

2.3.1 Chamada de função

Uma chamada de função seguido de um ponto e vírgula, passa a ser uma instrução válida da linguagem Fava, se essa função estiver declarada como não retornando valor. Se a função for definida como retornando um valor, então deixa de ser uma instrução válida. Por exemplo, mediante as declarações de funções mencionadas anteriormente, apenas a 1ª linha do seguinte excerto de código corresponde a uma instrução sintaticamente válida:

```
xpto(x,y);    // é válido porque xpto não retorna valor
fatorial(5);  // não é válido porque fatorial retorna um valor
```

Note porém, que `fatorial(5)` é considerado uma expressão válida (ver Secção 2.4 adiante).

Return

Esta instrução permite terminar a execução de uma função, retornando um valor que deverá ser do mesmo tipo que foi especificado na definição da função. A sintaxe da instrução é a palavra reservada `return` seguida de uma expressão, seguida de um ponto-e-vírgula. A expressão é opcional, porque podemos querer terminar a execução de uma função sem retornar qualquer valor. Este comportamento é análogo ao que acontece em C e Java. Exemplos:

```
return a+b;
return ;
```

Bloco (alteração)

A sintaxe de um bloco passa a ser: chaveta a abrir, seguido de zero ou mais declarações de variáveis, seguido de zero ou mais instruções, seguido de chaveta a fechar. Exemplo:

```
{
    integer i := 0;
    a := i + 1;
    b := a * a;
}
```

2.4 Novo tipo de expressão

Chamada de função

Uma chamada de função passa a ser uma expressão válida. Uma chamada de função é especificada tal como em C e Java: nome da função, seguido de um parêntesis a abrir, seguido de uma lista (porventura vazia) de argumentos separados por vírgula, seguido de um parêntesis a fechar. Cada argumento, por sua vez, é também uma expressão. Exemplo de expressão válida:

```
fatorial(5)
```

Durante a verificação de tipos, deve ser feita também a verificação de consistência entre o tipo de dados dos argumentos/parâmetros formais e dos argumentos/parâmetros actuais, bem como do seu número. No exemplo dado, a função fatorial foi declarada como tendo 1 argumento do tipo integer. Logo, quando a função é chamada terá de ser garantido que lhe estamos a passar uma (e apenas uma) expressão, cujo tipo tem de ser consistente com o tipo declarado na definição da função.

Se a passagem do parâmetro for feita por valor, em tempo de execução uma cópia do valor do argumento é copiado para o parâmetro formal da função.

Se a passagem for feita por referência, então o argumento tem de ser forçosamente uma variável, e o que é passado é o endereço da variável.

3 Máquina Virtual F

De modo a suportar a execução de funções, a máquina virtual vai passar a ter um registo especial chamado FP (Frame Pointer) que aponta para a base do *frame* em execução. Para além disso devem ser acrescentadas 7 novas instruções à máquina virtual, todas elas contendo um argumento inteiro. O nome e descrição destas novas instruções é apresentado na tabela abaixo.

Opc	Nome	Arg.	Descrição
46	lalloc	inteiro n	<i>local memory allocation</i> : aloca n posições no topo do stack para armazenar variáveis locais. Essas n posições de memória ficam inicializadas com o valor NIL.
47	lload	inteiro $addr$	<i>local load</i> : empilha o conteúdo de $Stack[FP + addr]$ no stack.
48	lstore	inteiro $addr$	<i>local store</i> : faz pop do stack e guarda o valor em $Stack[FP + addr]$
49	pop	inteiro n	desempilha n elementos do stack

50	call	inteiro <i>addr</i>	Cria um novo frame no stack, que passará a ser o frame corrente. Para tal terá de guardar o estado da máquina virtual (isto é, empilha o valor do registo <i>FP</i> , actualiza o valor de <i>FP</i> para apontar para a base do novo frame, e empilha o endereço de retorno: o endereço da instrução imediatamente após o call). Depois actualiza o <i>instruction pointer IP</i> de modo a que a próxima instrução a ser executada seja aquela que se encontra na posição <i>addr</i> do array de instruções.
51	retval	inteiro <i>n</i>	<i>return from non-void function</i> : faz $x = pop()$, desempilha o espaço reservado para as variáveis locais usadas pela função, restaura o estado da máquina virtual, desempilha os <i>n</i> argumentos do stack, e depois empilha <i>x</i>
52	ret	inteiro <i>n</i>	<i>return from void function</i> : desempilha o espaço reservado para as variáveis locais usadas pela função, restaura o estado da máquina virtual, e desempilha os <i>n</i> argumentos do stack

4 Reporte de erros

Tal como no 2º trabalho, vamos querer reportar os eventuais erros semânticos que possam haver. As mensagens de erro devem indicar a linha onde o erro ocorre, seguido de mensagem apropriado. As mensagens de erro devem vir ordenadas pelo número da linha onde o erro ocorre. (Ver exemplos de casos de teste de inputs/outputs que acompanham este enunciado).

À semelhança dos trabalhos anteriores, no caso de haver erros lexicais ou de parsing, o programa deve limitar-se a dizer `Input has lexical errors` ou `Input has parsing errors`, consoante o caso, e terminar a execução.

5 Requisitos

- O trabalho deve ser feito em Java usando o ANTLR 4, e submetido ao mooshak.
- Devem submeter um ficheiro zip que deverá conter uma pasta chamada `src` onde está todo o código que desenvolveram.
- A gramática deve chamar-se `Fava.g4` e deverá estar na pasta `src`
- A pasta `src` deverá conter o código gerado pelo ANTLR, supostamente numa pasta/package chamada `Fava`

- Nota importante: NÃO DEVEM incluir o ficheiro `antlr-4.13.2-complete.jar` na vosso zip. O servidor do mooshak já lá tem esse ficheiro.
- No código apenas pode haver um ficheiro java que tenha o método `main`. Não pode haver mais nenhum `main`, mesmo que esteja comentado. O nome do ficheiro que tem o método `main` deve chamar-se `FavaCompileAndRun.java`
- O vosso código é submetido ao mooshak, pelo que deverá poder ler o input a partir do *standard input*. Não obstante, o código deve estar também preparado para poder receber o input (nome do ficheiro Fava a compilar, e eventuais flags) a partir de argumentos passados à função `main`.
- O método `main` deverá definir as seguintes 2 flags que servem para controlar o modo como a emissão de erros é feita.

```
boolean showLexerErrors = false;
boolean showParserErrors = false;
```

Quando o código é submetido ao mooshak, as flags devem ter o valor `false` de modo a não mostrar as mensagens de erro. Porém, se o valor das flags for alterado para `true`, o vosso programa é suposto emitir os respectivos erros com mensagens apropriadas.

- Para efeitos de submissão ao mooshak:
 - se o input tiver erros lexicais, o programa deverá enviar a seguinte mensagem para o output e terminar: `Input has lexical errors`
 - se não tiver erros lexicais, mas tiver erros de parsing, o programa deverá enviar a seguinte mensagem para o output e terminar: `Input has parsing errors`
 - se não tiver erros lexicais nem de parsing, mas tiver erros semânticos, o programa deverá enviar mensagens de erro apropriadas para o output e terminar (ver exemplos).
 - se não tiver erros lexicais, nem de parsing, nem semânticos, o programa deverá gerar bytecodes de acordo com a especificação da máquina virtual e guardá-los num ficheiro com o nome `bytecodes.bc`. Após ser gerado, esse ficheiro deverá ser lido de imediato e a máquina virtual deverá executar os bytecodes.

6 Sobre o Mooshak

- <http://deei-mooshak.ualg.pt/~flobo/>, concurso Comp26, problema C.
- Não use caracteres acentuados no código, nem mesmo nos comentários, uma vez que o mooshak poderá dar erros por causa disso.

- O compilador de Java instalado no Mooshak é o openjdk versão 21. Se no vosso desenvolvimento usarem um JDK mais recente, recomendo que configurem o vosso IDE para não usar funcionalidades do Java posteriores à versão que está no Mooshak. (Se usarem o IntelliJ vão a 'Project Structure' → 'Language Level', e escolham 21.

7 Validação e avaliação

Os trabalhos serão validados e avaliados após a discussão individual a ter lugar nos dias 19, 21, 22, e 26 de Maio, em horário que será anunciado através da tutoria eletrónica.

Os trabalhos serão avaliados de acordo com a clareza e qualidade do código implementado, pela correcta implementação do compilador e máquina virtual, pelo cumprimento dos requisitos acima enunciados, e pelo desempenho individual durante a validação/discussão.

8 Prazo de entrega

O trabalho deve ser realizado em grupo, de acordo com as inscrições em grupo escolhida na tutoria. Deverão submeter o vosso código ao mooshak até às 23:59 do dia 17/Mai/2026. (**NOTA:** Não serão aceites entregas fora de prazo, nem que seja por um só minuto. Não devem deixar a submissão para os últimos instantes. Podem submeter o vosso trabalho várias vezes. Apenas será considerada a última submissão que tiver 'Accepted', ou no caso de não terem Accepted contará a última submissão.